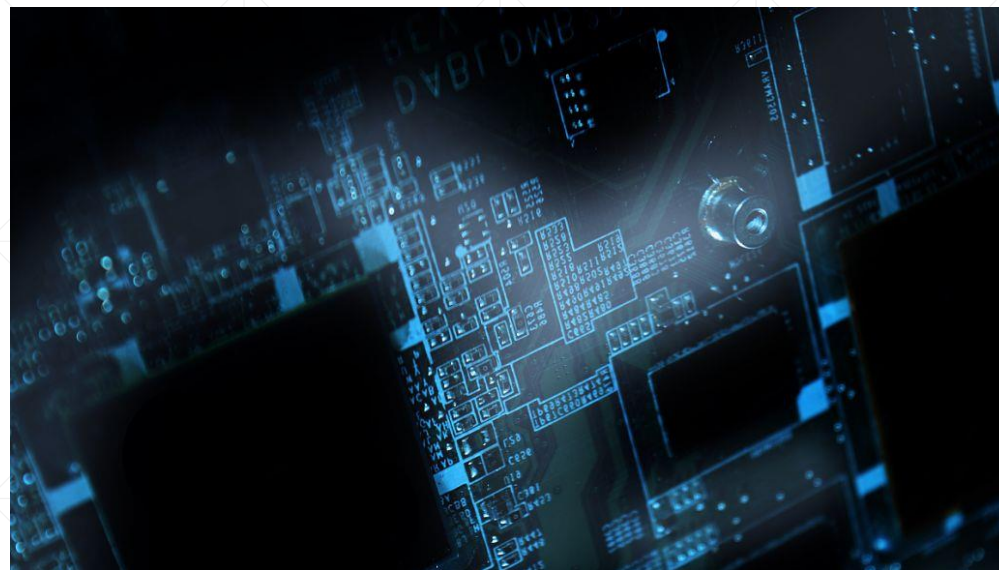
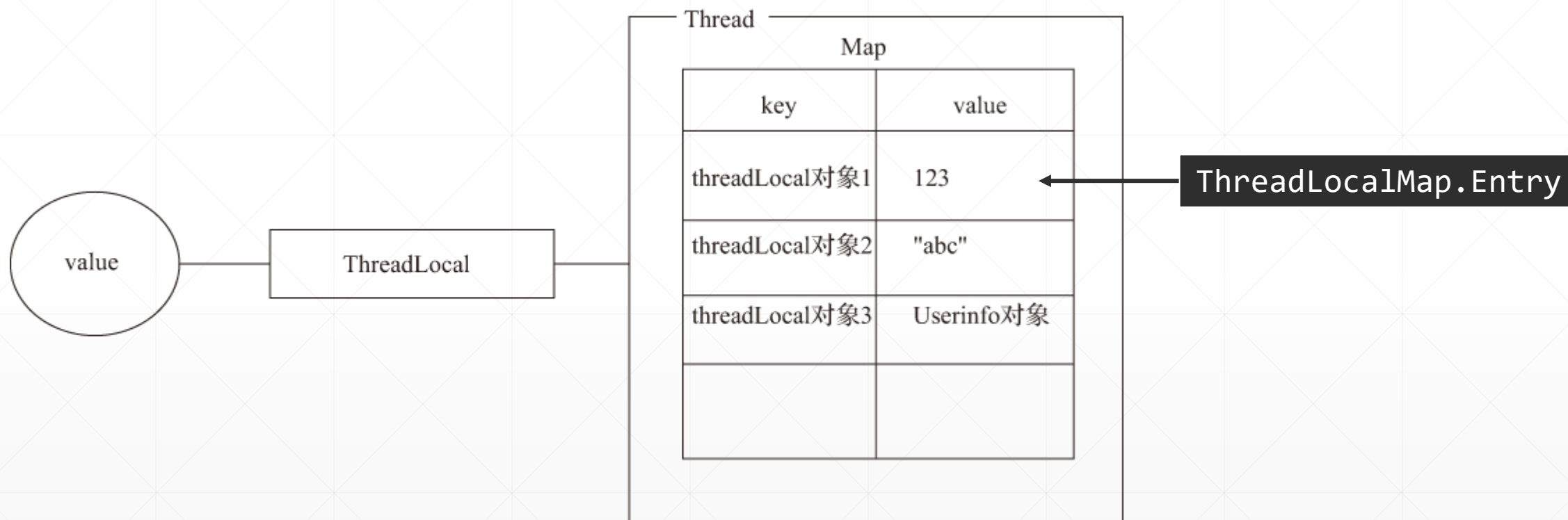




了解ThreadLcoal的技术特性

北京理工大学计算机学院
金旭亮

ThreadLocal的内部结构



每个Thread对象，都包容着一个ThreadLocalMap对象，其中，以ThreadLocal对象为Key，一对一地保存着数据。

ThreadLocal的主要方法



`T initialValue()`: 初始化，默认情况下，返回`null`。



`void set(T value)`: 设置一个新值。



`T get()`: 得到当前值，默认情况下，如果以前没有设置过，将得到一个`null`值。



`void remove()`: 清除数据。如果尝试再次读取，这个数据会被重新初始化（即调用`initialValue()`方法再初始化一次）。



使用完`ThreadLocal`之后，应该调用`remove`方法将`value`值置为`null`，以避免发生内存泄露。

get方法

此方法访问当前线程内部的ThreadLocalMap，调用map.getEntry方法，传入一个ThreadLocal引用作为Key，到Map中去查找值。



ThreadLocalMap是保存在线程中，而不是ThreadLocal中，因此，get方法返回的值，是Map中对应Key的值。

initialValue()方法



默认情况下，如果没有向ThreadLocal中放置值，这时，调用它的get()方法，会得到一个null值，这个值是initialValue()方法的返回值。



如果确实需要有一个默认值，则可以自定义一个类，派生自ThreadLocal，然后，重写它的initValue()方法即可。



通常每个线程最多调用一次这个方法，不过，如果先remove()掉原始值，再调用get()方法尝试读取，则会再一次调用此方法。

//一个支持自定义初始值的ThreadLocal类

2 usages

```
 class MyThreadLocal<T> extends ThreadLocal<T> {  
    2 usages  
    private final T initialValue;  
  
    1 usage  
     public MyThreadLocal(T initialValue) {  
        |     this.initialValue = initialValue;  
     }  
  
    @Override  
     protected T initialValue() {  
        |     return initialValue;  
     }  
     }
```

```
public class DefaultValueDemo {  
    public static void main(String[] args) {  
        MyThreadLocal<String> threadLocal = new MyThreadLocal<>(initValue: "这是我指定的默认值");  
        Runnable runnable = () -> {  
            var threadName : String = Thread.currentThread().getName();  
            //先提取默认值  
            System.out.println(threadName + ":" + threadLocal.get());  
            //再设置自己的值  
            threadLocal.set(threadName + "设置值:" + new Random().nextInt());  
            //再次提取值  
            System.out.println(threadName + ":" + threadLocal.get());  
        };  
        new Thread(runnable).start();  
        new Thread(runnable).start();  
    }  
}
```

Run: DefaulValueDemo x

"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"

Thread-0:这是我指定的默认值

Thread-1:这是我指定的默认值

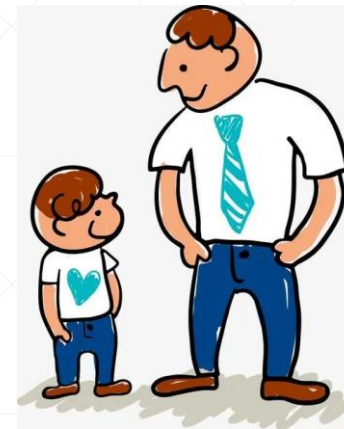
Thread-1:Thread-1设置值:728221457

Thread-0:Thread-0设置值:-248024675

Process finished with exit code 0

父子线程

通常情况下，使用`ThreadLocal`是为了存储那些每个线程都独享的数据，但在某些特定场景下，也可以将一个数据从一个线程传给另一个线程。



JDK提供了一个`InheritableThreadLocal`类型，可以实现父线程将自己的`ThreadLocal`内容复制一份，发给子线程。



这种父子线程间的数据传送，是“单向”的，“仅一次”的。


```
UseInheritableThreadLocal.java x
1 public class UseInheritableThreadLocal {
    5 usages
2     private static final InheritableThreadLocal<String> threadLocal =
3         new InheritableThreadLocal<>();
4
5     public static void main(String[] args) throws InterruptedException {
6         threadLocal.set("这是父线程设置的值");
7         var thread = new Thread(() -> {
8             System.out.println("子线程读到: " + threadLocal.get());
9             threadLocal.set("子线程设置的值");
10            System.out.println("子线程读到: " + threadLocal.get());
11        });
12        thread.start();
13        thread.join();
14        System.out.println("父线程读到: " + threadLocal.get());
15    }
16 }
```

可以看到，子线程只是在启动时“接收”了从父线程传来的数据，但以后，父子线程的数据，就互不影响了。

```
Run: UseInheritableThreadLocal x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
子线程读到: 这是父线程设置的值
子线程读到: 子线程设置的值
父线程读到: 这是父线程设置的值
Process finished with exit code 0
```